

Presenter notes

Slides 01-02 / Read the Say text; use optional detail only when needed.

PRESENT SLIDES 1-12 ONLY

The spoken script targets 5:15. Add short pauses and stay within 6 minutes. Slides 13-19 are reference appendices. Optional details are outside the timed script.

SLIDE 01

Target 9 sec | 0:00 - 0:09

TengeFlow

SAY

Hello, I am Pinigin Artyom from BDA-2602. TengeFlow helps students track everyday expenses and understand their budget.

DETAIL FOR QUESTIONS / OPTIONAL

The project follows Topic 35, Personal Finance: Personal Expense Tracking Application.

SLIDE 02

Target 32 sec | 0:09 - 0:41

The problem and the person

SAY

Small purchases are easy to remember separately, but harder to understand as a monthly total. Ayan is our fictional twenty-year-old student. He uses his phone between classes and pays with both cash and cards. He needs a quick way to record purchases and see what remains. TengeFlow brings those records together without connecting to a bank account.

DETAIL FOR QUESTIONS / OPTIONAL

Ayan is a design persona, not an interviewed participant. Future user testing should check these assumptions.

Presenter notes

Slides 03-04 / Read the Say text; use optional detail only when needed.

SLIDE 03

Target 22 sec | 0:41 - 1:03

Five screens, one clear purpose

SAY

There are five core finance screens. Overview gives the summary. Add Expense is the primary task. Transactions supports finding and correcting records. Budgets shows limits, and Insights explains spending patterns. All five use the same records, so one change updates the relevant totals throughout the application.

DETAIL FOR QUESTIONS / OPTIONAL

The landing page, registration and onboarding support the main five-screen application.

SLIDE 04

Target 22 sec | 1:03 - 1:25

From registration to a personal budget

SAY

New users register and confirm their email. Then they set their name, monthly budget and five category limits. The limits must fit the budget. The server saves the setup together, and the account begins with an empty history. Returning users open their existing workspace directly.

DETAIL FOR QUESTIONS / OPTIONAL

Development uses a local email inbox. Public account access will require a hosted backend and real SMTP email delivery.

Presenter notes

Slides 05-06 / Read the Say text; use optional detail only when needed.

SLIDE 05

Target 40 sec | 1:25 - 2:05

A purchase becomes budget feedback

SAY

This is the separate demo, using sample data. We start with a budget of one hundred and twenty thousand tenge. I enter a food purchase of three thousand five hundred, select the payment method and save. The confirmation shows that the record was accepted. Spending increases by that amount, and the remaining budget decreases. The exact before and after totals are visible on the slide.

DETAIL FOR QUESTIONS / OPTIONAL

Point to each state while speaking. Before: budget ₸120,000, spent ₸77,650, left ₸42,350. After adding ₸3,500: spent ₸81,150 and left ₸38,850. New real accounts do not contain this demo data.

SLIDE 06

Target 24 sec | 2:05 - 2:29

Review spending and adjust the plan

SAY

Users can search their history, filter records, correct an amount or confirm a deletion. Budgets compares spending with category limits and makes overspending visible. Insights shows where money goes and how spending changes over time. These views help users review their choices using the same underlying transactions.

DETAIL FOR QUESTIONS / OPTIONAL

Explain one pattern visible in the chart as a pattern in the sample data. Detailed history and budget screenshots are in the appendices.

Presenter notes

Slides 07-08 / Read the Say text; use optional detail only when needed.

SLIDE 07

Target 24 sec | 2:29 - 2:53

Mobile interaction and recovery

SAY

On a phone, bottom navigation and large controls make the main actions easier to reach. Labels and nearby error messages explain what needs correction. Drafts preserve unfinished input. Dialogs open smoothly, support keyboard focus and respect reduced-motion preferences. The goal is a clear task with an easy way to recover from mistakes.

DETAIL FOR QUESTIONS / OPTIONAL

The app has a manifest and icons, but no service worker. Do not describe it as guaranteed to load offline.

SLIDE 08

Target 42 sec | 2:53 - 3:35

System architecture

SAY

The phone or laptop runs the interface in its browser. React displays the screens, while Zustand keeps the current state. A repository connects this interface to either the local demo or our Flask backend. For accounts, the backend applies the rules and stores records through SQLAlchemy in SQLite. The planned hosted version uses Vercel for the frontend, with a separate API, persistent database and email service.

DETAIL FOR QUESTIONS / OPTIONAL

Follow the diagram from device to database. The local account architecture is implemented; the public hosted route is a deployment plan. PostgreSQL is a supported future database option.

Presenter notes

Slides 09-10 / Read the Say text; use optional detail only when needed.

SLIDE 09

Target 30 sec | 3:35 - 4:05

One complete data transaction

SAY

When an account user saves an expense, the client checks the fields and sends a JSON request. The server checks the session, validates the input and identifies the owner. It then saves the record in the database. Only after a valid success response does the interface update its totals. An error keeps the draft instead of showing false success.

DETAIL FOR QUESTIONS / OPTIONAL

If the response is lost, the write might already have succeeded. Refresh the history before retrying. Idempotent expense submission is future work.

SLIDE 10

Target 34 sec | 4:05 - 4:39

Mobile and cloud trade-offs

SAY

A hosted account could make records available on different devices over Wi-Fi, 4G or 5G. It also depends on connectivity, hosting and data protection. A fully local app is simpler, but its data stays in one browser. Here, account saves require a connection. The demo only simulates synchronisation. Our shared repository keeps these storage choices separate from the interface.

DETAIL FOR QUESTIONS / OPTIONAL

Local storage can be cleared accidentally. Cloud recovery also requires configured and tested backups; hosting alone does not guarantee recovery.

Presenter notes

Slides 11-12 / Read the Say text; use optional detail only when needed.

SLIDE 11

Target 26 sec | 4:39 - 5:05

Limitations and next development

SAY

The current version uses manual entry, one currency and five fixed categories. It has no bank connection or real account offline queue. Next steps are hosted account access with real email, reliable retries and usability testing. Recording an expense in fifteen seconds is a design target, not a measured result.

DETAIL FOR QUESTIONS / OPTIONAL

Additional ideas include recurring expenses, CSV export and per-month budget history. First, validate the core flow with real users.

SLIDE 12

Target 10 sec | 5:05 - 5:15

Ready to demonstrate

SAY

TengeFlow connects everyday purchases with understandable budget feedback. I am ready to demonstrate the main flow and answer your questions.

DETAIL FOR QUESTIONS / OPTIONAL

Demonstration order: overview, add expense, confirmation, history, budgets and insights. Use the separate local demo if live account services are unavailable.

Questions to prepare for

Optional answers / These are not part of the timed presentation.

QUESTION 01

Why does this application need a backend?

The backend stores account records independently of one browser, verifies sessions and controls access to each user's data. The separate demo can work locally, but it cannot provide the same account service.

QUESTION 02

What is the difference between authentication and authorization?

Authentication verifies the user's identity. Authorization decides what that user may access. TengeFlow checks the session and restricts financial queries to the authenticated owner.

QUESTION 03

What happens when the internet connection is lost?

Account saves need a connection. The interface keeps the draft and does not report an unconfirmed save as successful. The local demo can store pending entries, but its synchronisation status is simulated and does not upload data.

Questions to prepare for

Optional answers / These are not part of the timed presentation.

QUESTION 04

What if the server saves an expense but its response is lost?

The expense may already exist. The user should refresh the history before retrying. A future idempotency key would let the server recognise a repeated submission and prevent duplicate creation.

QUESTION 05

Why choose Flask and SQLite, and can the system grow?

Flask provides a straightforward Python API, and SQLite simplifies local setup. SQLAlchemy keeps the database models separate from the interface. PostgreSQL is a supported deployment option, although a public deployment still needs configuration and testing.

QUESTION 06

Is TengeFlow publicly hosted and fully usable offline?

No public deployment is claimed. The frontend can be hosted on Vercel, but working accounts also need the hosted API, persistent storage and real email delivery. A service worker and genuine account offline synchronisation are not implemented.

LOCAL DEMO PREPARATION

From the project folder, run `npm run backend:start` and `npm start`. Open `http://localhost:3000/demo` and use sample data. Start with a fresh demo so the totals match the slides.

Public project link: pending deployment. No live public account service is claimed.